

In theory, there is no diff b/w theory & practice, but in practice it is.

camlin

Date: 14/01/2026

Textbooks:

Andriy Burkov, Machine Learning Engineering, True Positive Inc., 2020.

Noah Gift, and Alfredo Deza, Practical MLOps:

Operationalizing Machine Learning Models, O'Reilly, 2021.

AutoML $\rightarrow$ SDL topic (in textbook 2).

Scalar - is a simple numerical value.

- represent single quantities & are the simplest form of data used in maths expression.

Vector - is an ordered list of scalar values, called attributes.

- denoted using bold characters.

- viewed in 2 ways.

- Attribute $\rightarrow$ written as italic letter with an index. Eg: $w(i)$, $x(i)$.

- Index

Matrix - vector relationship.

Eg: Feature 1: Size (sqft)

Feature 2: No. of bedrooms

Feature 3: Age of house (years)

100	1	5
500	2	8
1500	4	10
2000	5	14

$\downarrow$
Feature 1

2

3

Sets - unordered collection of unique elements
- denoted using calligraphic capital letters

- Represented using braces.

- Eg: $\{1, 2, 3\}$ - finite set

- Infinite sets: Defined using intervals.

$[a, b]$ - includes a & b .

(a, b) - excludes a & b .

Real number set $\rightarrow$ includes all no.s from $-\infty$ to $+\infty$

Membership $\rightarrow x \in S$.

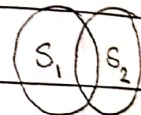
Intersection: $\cap$; Union: $\cup$

Set size: $|S|$

Let S_1 & S_2 be 2 finite sets.

Prove that: $|S_1 \cup S_2| + |S_1 \cap S_2| = |S_1| + |S_2|$

Proof: $(|S_1| + |S_2| - |S_1 \cap S_2|) + |S_1 \cap S_2|$
 $= |S_1| + |S_2|$



Proof: Partition the sets as

$S_1 \setminus S_2$: Elements only in S_1 ($S_1 - S_2$)

$S_2 \setminus S_1$: Elements only in S_2

$S_1 \cap S_2$: Elements in both S_1 & S_2

The 3 sets above are disjoint.

$$|S_1 \cup S_2| = |S_1 \setminus S_2| + |S_2 \setminus S_1| + |S_1 \cap S_2| \quad \text{--- (A)}$$

$$\text{Wkt, } |S_1| = |S_1 \cup S_2| - |S_2 \setminus S_1| + |S_1 \cap S_2| \quad \text{--- (1)}$$

$$|S_2| = |S_1 \cup S_2| - |S_1 \setminus S_2| + |S_1 \cap S_2| \quad \text{--- (2)}$$

$$\text{(1) + (2)} \Rightarrow |S_1 \setminus S_2| + |S_1 \cap S_2| + |S_2 \setminus S_1| + |S_1 \cap S_2|$$

$$= |S_1 \setminus S_2| + |S_2 \setminus S_1| + 2|S_1 \cap S_2| + |S_1 \cap S_2|$$

Substitute A in B.

$$|S_1| + |S_2| = |S_1 \cup S_2| + |S_1 \cap S_2|$$

Dataset overlap:-

S_1 = users in training dataset

S_2 = users in test dataset.

$|S_1| + |S_2|$ counts users in both datasets, but overcounts users that accidentally appear in both (data leakage).

Using: $|S_1 \cup S_2| + |S_1 \cap S_2| = |S_1| + |S_2|$

can be used to detect data leakage.

Model Prediction:-

S_1 = samples predicted +ve by Model A

S_2 = samples predicted +ve by Model B.

Then, $S_1 \cap S_2 \rightarrow$ high-confidence predictions.

$S_1 \cup S_2 \rightarrow$ all positives predicted by at least one model.

This logic is used in ensemble methods, consensus-based filtering, weak supervision.

Model Prediction: Binary Classification:-

Email Spam detection.

Total emails = 10.

Task: classify each email as Spam or NotSpam.

Step 1: Define sets.

S_1 = emails predicted as spam by model A

S_2 = emails predicted as spam by model B

$$S_2 = \{e_2, e_3, e_4, e_7\}$$

Step 2: Intersection (Agreement)

S_1 = emails predicted as spam by model
 S_2 = " " " "

$|S_1 \cap S_2|$ = means agreement b/w 2 models

$$\begin{aligned} |S, \cup S_2| + |S, \cap S_2| &= |S_1| + |S_2| \\ |S, \cup S_2| &= |S_1| + |S_2| - |S, \cap S_2| \end{aligned}$$

If 2 spans

Capital Sigma Notation :-
Examples where we are applying :-

$$MSE = \frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2$$

total error across dataset
we want to minimize this

Gradient Descent :

$$\frac{\partial L}{\partial w} = \sum_{i=1}^n x_i (\hat{y}_i - y_i)$$

used in optimisations, model training

Linear Regression :-

$$\hat{y} = \sum_{j=1}^d w_j x_j$$

weighted feature sum

$$P_y = \sum_x P(y/x) P(x)$$

Naive Bayes classifier

classification accuracy

$$Acc = \frac{1}{n} \sum_{i=1}^n 1(\hat{y}_i = y_i)$$

Returns 1/0 $\Rightarrow$ counts correct predictions.

L2 Regularisation

$$\lambda \sum_{j=1}^d w_j^2$$

Penalizes large weight.

NN Loss + Backpropagation

$$L = \sum_{i=1}^n \text{loss}_i$$

What is Machine Learning :-

↳ behavior of algorithm is not explicitly programmed.
Instead, it is inferred from data.

Types of ML :- supervised, unsupervised etc.

Dataset representation :

$$X = \begin{bmatrix} x_{11} \\ \vdots \\ x_{nm} \end{bmatrix} \quad y = \begin{bmatrix} y_1 \\ \vdots \\ y_n \end{bmatrix}$$

Learning objective :

$$f : X \rightarrow y$$

$$\hat{y} = f(x)$$

Model :

$$\hat{y} = w^T x + b$$

$$\text{Loss function} = \frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2$$

Problem 1 :

Binary Classification :

Features — Avg. order value last 30 days

— Time since last purchase

— Discount usage

— App session count

— Customer support information

Learning objective :

$$f(x) = P(\text{churn})$$

$y : 0$ — customer stayed active

$y : 1$ — customer churned in 60 days.

Model — Logistic Regression / Gradient Boosted tree.

Loss function: Binary cross entropy.

Goal is to minimize churn misclassification.

↑ Ques: An e-commerce company notices that customers who previously made large, frequent purchases suddenly become inactive without complaints or churn signals. The business wants to identify at-risk customers early so retention offers can be triggered proactively.

Problem 2:

A factory sees machines passing quality checks but failing soon after deployment, causing warranty claims. Management wants to predict early-life failure risk before shipping.

Features.

Regression :

Feature: — Sensor Reading
— Batch ID
— Assembly time
— Duration of test.

Learning objective :

predict y (Time to failure)

$y \rightarrow$ Predict #days to first failure

Model :

Loss Function : MSE

Goal is to minimize error & has high risk

Problem 3:

A bank observes that customers with acceptable credit scores still default, especially under changing economic conditions. The goal is to improve default prediction beyond credit score.

Features : — bank balance
— Credit score
— transaction history

$y \rightarrow 1$: defaulted.

0 : fully repaid.

Goal $\rightarrow$ Reduce FN

False Negative

(predicted safe but customers defaulted)

Problem 4

A SaaS company sees users actively using the product during trial but not converting to paid plans. The business wants to predict conversion likelihood & intervene with targeted nudges.

Features:

Learning objective

Model:

Loss funct.

Goal

Problem 5:

A logistics firm meets SLA targets on average, yet some routes fail unpredictably causing customer dissatisfaction. They want to predict delivery delay duration before dispatch.

Features: - Distance
- Weather
- Time of dispatch
- Driver experience
- Warehouse congestion

Predict y

y : delay in days

Loss function: MSE.

Goal is to minimize avg. delay prediction.
(error).

(Classification vs Regression)

Supervised learning libraries:

Algo	Typ	Main library	importing lib / class
Linear Regression	Reg.	scikit-learn	from sklearn.linear - model import LinearRegression.
Logistic Reg.	Clas.	"	from sklearn.linear - model import LogisticRegression.
KNN	clas. / Reg.	"	from sklearn.neighbors import KNeighborsClas- sifier / KNeighbors Regressor
Decision Tree	Clas. / Reg.	"	from sklearn.tree import DecisionTree Classifier / Decision Tree Regressor.
Random Forest	clas. / Reg.	"	from sklearn.ensemble import RandomForest Classifier / RandomForest Regressor.
Support Vector Machine (SVM)	"	"	from sklearn.svm import SVC / SVR
Naive Bayes	Clas.	"	from sklearn.naive- bayes import Gaussian NB
Neural Networks (MLP)	Clas. / Reg.	"	from sklearn.neural- network import MLPClassifier MLP Regressor.

XGBoost	class/	XGBoost	from xgboost import
	Reg.		XGBClassifier / XGBRegressor
DNN	"	TensorFlow/	import tensorflow as
		PyTorch	tf or import torch

ACCURACY :

$$\text{Accuracy} = \frac{TP + TN}{TP + TN + FP + FN}$$

When to use → classes are balanced
 → cost of false positive & false negative is similar.

When not to use → Imbalanced data (eg: churn, fraud)
 → When one type of error is ^{more} costly.

PRECISION ≠ SPECIFICITY.

(When the model said a YES, how often is it correct)

→ Precision = $\frac{TP}{TP + FP}$

What it controls → false positive

When to use → Action are expensive.
 → False alarms are costly.

Of all actual negatives, how many did we correctly reject?

$$\rightarrow \text{Precision} = \frac{TN}{TN + FP} \quad (\text{Specificity})$$

Rejecting a good case is expensive.

RECALL (Sensitivity).

(Out of all actual YES cases, how many did we catch?)

$$\text{Recall} = \frac{TP}{TP + FN}$$

What it controls $\rightarrow$ False negatives.

When to use $\rightarrow$ Missing a positive case is costly.

F1 - SCORE

A single score that balances precision & recall.

$$F1 = 2 \cdot \frac{\text{Precision} \cdot \text{Recall}}{\text{Precision} + \text{Recall}}$$

$\rightarrow$ is high only if both precision & recall is high.

SUPPORT

(how many actual samples belong to each class)

$\rightarrow$ it is a count.

Confusion matrix:-

		Predicted	
		Positive	Negative
Actual	Positive	TP	FN
	Negative	FP	TN

Steps in all libraries:-

1. Define the model
2. Provide labeled data (X, y)
3. Choose a loss function (explicitly or implicitly)
4. Train the model
5. Evaluate predictions.

Que: An e-commerce company wants to predict whether a customer will churn in the next 60 days so that retention actions can be taken early.

$$y = \begin{cases} 1 & \text{customer churns.} \\ 0 & \text{customer stays. (not churned)} \end{cases}$$

Write code to implement this via Logistic Regression & xgboost with appropriate libraries. Your data is in customer_churn.csv.

Based on your implementations, pick a model & interpret the results.

Comment on how you are able to help the company.
 Ans: Logistic Regression :- (Scikit-learn)

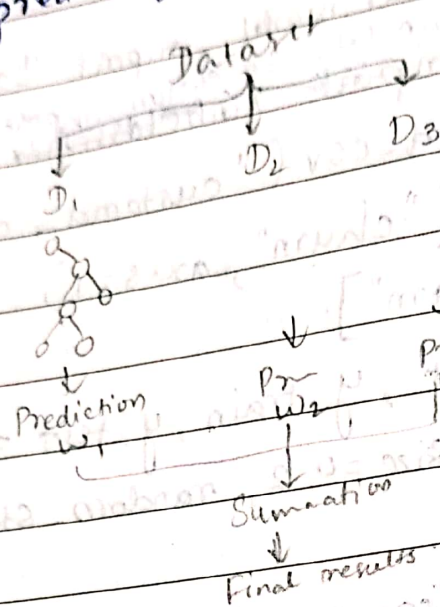
```
import pandas as pd
from sklearn.linear_model import LogisticRegression
from sklearn.model_selection import train_test_split
from xgboost import XGBClassifier
data = pd.read_csv("customer_churn.csv")
X = data.drop("churn", axis=1)
y = data["churn"]
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42)

lr = LogisticRegression(max_iter=1000)
lr.fit(X_train, y_train)
y_pred_lr = lr.predict(X_test)
```

XGBoost : (Gradient Boosted Trees)

```
import pandas as pd
from xgboost import XGBClassifier
from sklearn.model_selection import train_test_split
data = pd.read_csv("customer_churn.csv")
X = data.drop("churn", axis=1)
y = data["churn"]
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42)
xgb = XGBClassifier(n_estimators=100, max_depth=
    =4, learning_rate=0.1, eval_metric="logloss",
    random_state=42)
```

$xgb.fit(X_{train}, y_{train})$
 $y_{pred} = xgb.predict(X_{test})$



Result:

Accuracy = 0.82

→ 82% of customers were correctly classified overall.

Class 0 (not churned)

→ Precision = 0.85

→ Among all customers the model predicted as "not churned", 85% truly did not churn.

The remaining 15% actually churned but were incorrectly measured.

Result Table:

Accuracy: 0.82

	Precision	recall	f1-score	support
0	0.85	0.88	0.86	320
1	0.75	0.68	0.71	180
accuracy			0.82	500
macro avg	0.80	0.78	0.79	500
weighted avg	0.82	0.82	0.82	500

Class 0 $\rightarrow$ Recall = 0.88

$\rightarrow$ Out of all customers who truly did not churn, the model correctly identified 88% of them. Only 12% of non-churners were mistakenly flagged as churners.

$\rightarrow$ Support = 320

$\rightarrow$ 320 non-churn customers in the test set.

Class 1 (Churned)

$\rightarrow$ Precision = 0.75

$\rightarrow$ When the model predicts churn, it is correct 75% of the time.

$\rightarrow$ Recall = 0.68

$\rightarrow$ It only catches 68% of actual churners (some are missed)

$\rightarrow$ Support = 180

$\rightarrow$ 180 churned customers in the test set

The model is conservative: It avoids false alarms but misses some at-risk customers.

Precision asks: When the model predicts a class, how often is it correct?

$\rightarrow$ High precision means few false alarms.

Recall asks: Of all truly relevant cases, how many did the model find?

$\rightarrow$ High recall means few misses.

XGBoost is (overall) better than Logistic Regression

The company wants to :

Identify customers who are about to churn early enough to intervene.

So the most costly mistake is :

A customer who was going to churn but the model did not flag.

RECALL OF CHURN helps with that.

Log. Regr. recall = 0.68.

→ Missed 32% of churners.

XGBoost recall = 0.82

→ Missed only 18% of churners.

Precision also matters

cost side -

Retention offers cost money.

LR precision = 0.75

→ 25% of offers wasted.

XGBoost precision = 0.85

→ only 15% wasted.

So XGBoost : Catches more churners,
wastes fewer offers.

MONEY SAVED!

Unsupervised Learning :-

Ques: An e-commerce company has customer data containing age, income, & spending behaviour. The company wants to segment customers into meaningful groups for marketing.

⇒ Discover groups or clusters in data.

⇒ K-means algo.

Ques: A city traffic dept. collects large volume of GPS location data from vehicles moving across the city over several days. Each record represents the position of a vehicle at a given time, & the dataset contains only GPS coordinates, with no labels such as road names, traffic zones, or congestion levels.

⇒ DB scan algo.

⇒ Discovers groups or clusters in data.

Ques: A medical dataset contains 300 diagnostic features collected from patient tests. There are no outcome labels, & the goal is to reduce the no. of features while retaining maximum information.

⇒ Autoencoders, PCA

⇒ Dimensionality Reduction.

Que: A Supermarket stores transaction records of items purchased together. There are no labels, & the goal is to identify frequently co-occurring products.

⇒ Discover hidden patterns / Associations in the data.

⇒ Apriori, FP growth

Que: An online learning platform tracks courses viewed & enrolled together by students. Platform wants to discover relationships b/w courses to improve recommendation.

⇒ Discover hidden patterns / associations.

⇒ Apriori, FP growth

Que: - - - - -

⇒ Detect anomalies / Outliers.

Goal	Algo.	Importing lib./class	Input (X)	O/p (Model result)
1. Discover groups / clusters	Kmeans	from sklearn.cluster import KMeans	Numeric feature Matrix $X = [\text{age, income, spend}, \dots]$	cluster labels for each data point $[0, 1, 2, \dots]$
	DBSCAN	from sklearn.cluster import DBSCAN	Numeric feature matrix (scaled) $X = [[x_1, x_2], \dots]$	cluster labels + noise (-1) $[0, 0, -1, 1, \dots]$

2. Reduce Dimensionality	PCA	from sklearn.decomposition import PCA	High dimensionality numeric matrix X ($n \times d$)	Reduced feature matrix X_{reduced} ($n \times k$)
	t-SNE	from sklearn.manifold import TSNE	High dimensional numeric matrix	2D or 3D embedding for visualization
3. Discovers patterns / association	Apriori	from mlxtend.frequent - patterns import apriori	Transaction matrix (binary) milk bread butter	Frequent itemsets & rules (support, confidence, lift)
4. Detect Anomalies / Outliers	Isolation Forest	from sklearn.ensemble import IsolationForest	Numeric feature matrix	Normal / anomaly labels $1 = \text{normal}$ $-1 = \text{anomaly}$
	One-class SVM	from sklearn.svm import OneClassSVM	Numeric feature matrix	Inlier / outlier prediction
5. Learn Representation	Auto encoder	import tensorflow as tf / import torch	Raw input data (images, signals, features)	Compressed latent representation + reconstructed input

Semi-Supervised Learning

(less amount of labelled data, large amount of unlabelled data).

→ Goal: Same as supervised learning.
i.e., produce a model that predicts label.

The assumption is that unlabelled data helps to reveal structure that improves learning accuracy.

Supervised $\rightarrow$ Teacher gives answers to every que.

Semi-supervised $\rightarrow$ Teacher gives few answers
Student predict rest of answers by seeing patterns.

① Goal $\rightarrow$ Improve classification with few labels

Algo $\rightarrow$ Pseudo-labelling

Type $\rightarrow$ Semi-supervised classification

Main library $\rightarrow$ SciKit-learn

Importing lib. $\rightarrow$ Uses base classifier.
(eg: Logistic regression)

I/p $\rightarrow$ Labeled X_l + Unlabeled X_u

O/p $\rightarrow$ Predicted labels for X_u .

② Goal $\rightarrow$ Improve classification with few labels.

Algo $\rightarrow$ Self-training

Type $\rightarrow$

Main lib $\rightarrow$

Importing lib $\rightarrow$ from sklearn. semi-supervised import
SelfTrainingClassifier.

I/p $\rightarrow$ small labelled set + large unlabelled ^{set} data.

O/p $\rightarrow$ Expanded labeled dataset.

③ Goal → Propagate labels.
 Algo → Label propagation.
 Type → Graph based semi-supervised.
 Main Lib →

Imp. lib → from sklearn.semi-supervised import LabelPropagation.

I/p → Feature matrix + partial labels

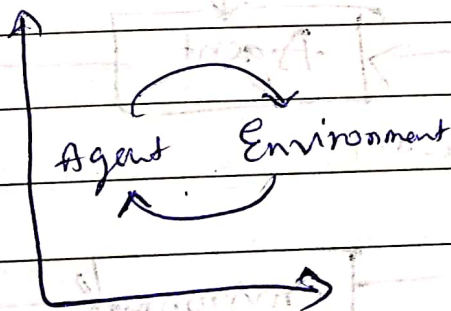
O/p → Predicted labels for unlabeled points.

④ Algo → Autoencoder &

Check!! → Diff. b/w pseudolabeling & self training & label propagation & Autoencoder + classifier.

↓
 for dimensionality reduction,
 Denoising, data compression.

Reinforcement Learning:-



→ Subfield of ML where a machine, known as an agent, "lives" within an environment.

Perception: The agent perceives the state of its environment as a vector of features.

Iterative process:-

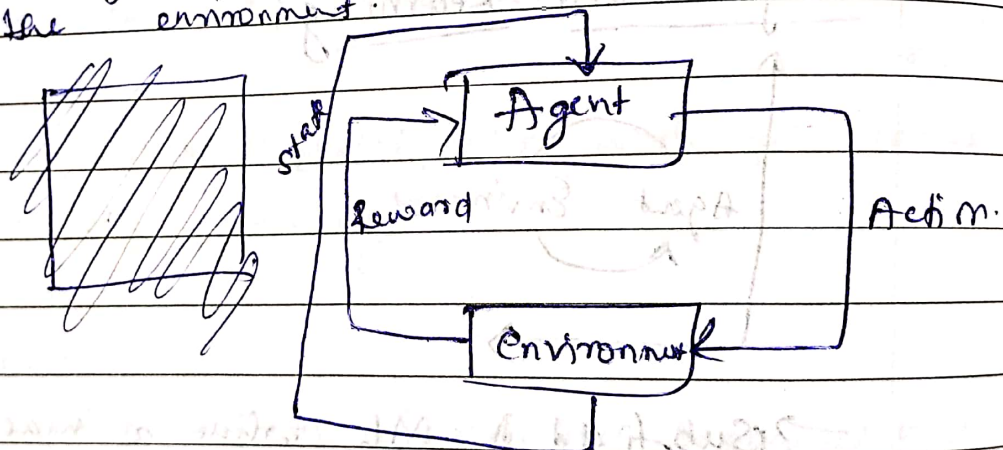
Sequential decisions: ultimate goal is long-term.

Eg: Self-learning tasks. (cars).

Core mechanics: State, Action, & Reward.

- State: representation of the current situation of the environment, provided to the agent as input.
- Action: The agent executes specific actions in non-terminal states to influence the environment.
- Reward: Different actions yield different numerical values which can be positive (incentive) or negative (penalty).

State transition: Executing an action moves the machine from its current state to a new state of the environment.



Optimal policy: Primary goal is to learn a func. that takes the feature vector of a state as i/p & o/p's the optimal action to execute in that state.

Maximizing Reward: An action is defined as optimal if it maximises the expected avg. longterm reward.

Exploration vs Exploitation

Similar to Multi-Armed Bandit problem, RL System must balance exploring the environment to find new rewards with exploiting known high-reward actions.

Types of R.L:

- 1) Value-Based RL
- 2) Policy-Based RL
- 3) Model-Based RL
- 4) Hybrid approaches: Actor-critic methods.

Case Study: AWS DeepRacer.

Initialize Environment
(Track, car position)

↓
Observe current state
(position, speed, wheels)

↓
Select Action
(steering, throttle) using
current policy

↓
Execute Action
Move car on track

Call Reward Function
 $\text{reward} = f(\text{params})$

↓
Update Policy
(Reinforcement Learning)

↓
Episode Finished?

↓ Yes
Reset
car

↓ No
Next Time
step

Date : _____

Goal : Encourage the car to stay close to the centre of the track.

Penalize the agent immediately if it goes off-track.

Multi-Armed Bandit (MAB) Problem

The MAB problem models a situation where an agent must choose among multiple actions (called arms) repeatedly, where:

Each arm gives a random reward.

Reward probabilities are unknown.

Goal : Maximize total reward over time.

Core characteristics :- No states, Decision is made repeatedly.

Must balance :

- Exploration : try different arms.
- Exploitation : use the best known arm.

A/B Testing : → 2 alternate testing.

→ statistical experiment where :

- Users are split into fixed groups.
- Each group receives a diff. variant.
- Performance is measured after sufficient data.
- One option is chosen at the end.

4/2/26 Use A/B Testing when:

- Changes are low frequency.
- Statistical confidence is required.
- Environment is stable.

5 Use MAB when:

- Decisions are repeated.
- User interaction is ongoing.
- You want to reduce regret in real time.

7 Eg: Ad or content selection.

Email subject line optimization.

15 Eg: Major product or UX redesign.

When you care about long-term metrics.

Case study: Uber.

→ Uses A/B testing across core business logic.

20 What they tested? → Surge pricing algorithms.

→ Driver incentive programs.

Why? → These affect revenue, fairness & user trust.

→ Decisions must be carefully validated before full rollout.

Case study : Udemy.

↳ uses MABs

→ Udemy built a real time bandit system for course recommendation.

→ uses MABs for ranking & recommendation.

Why ? → User preferences change constantly.

→ Need fast adaption, not just post-experiment analysis.

Data used directly and Indirectly :

← Data used directly —

Text data.

Can be represented as :

→ Raw tokens.

→ Word IDs

→ Embeddings (word2Vec, GloVe etc)

Data used Indirectly —

A "gazetter" is a precompiled list of domain specific names.

Some kind of preprocessing is done here.

Raw and Tidy Data

→ collection of entities in their natural form

Raw data — word docs, JPEG files or PDFs.

↳ unstructured or ~~was~~ semi structured.

↳ cannot be used directly by most ML algos.

Eg : Text, Image, Audio...
(PDF) (JPEG) (WAV file)

Tidy data — spreadsheet (each row → example
column → attribute)
↳ structured.

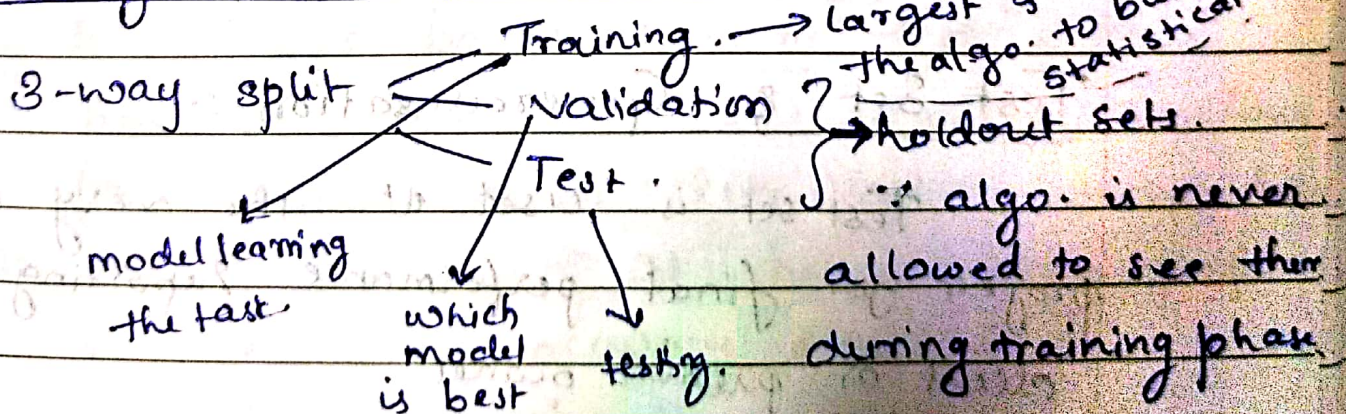
Eg : Email.

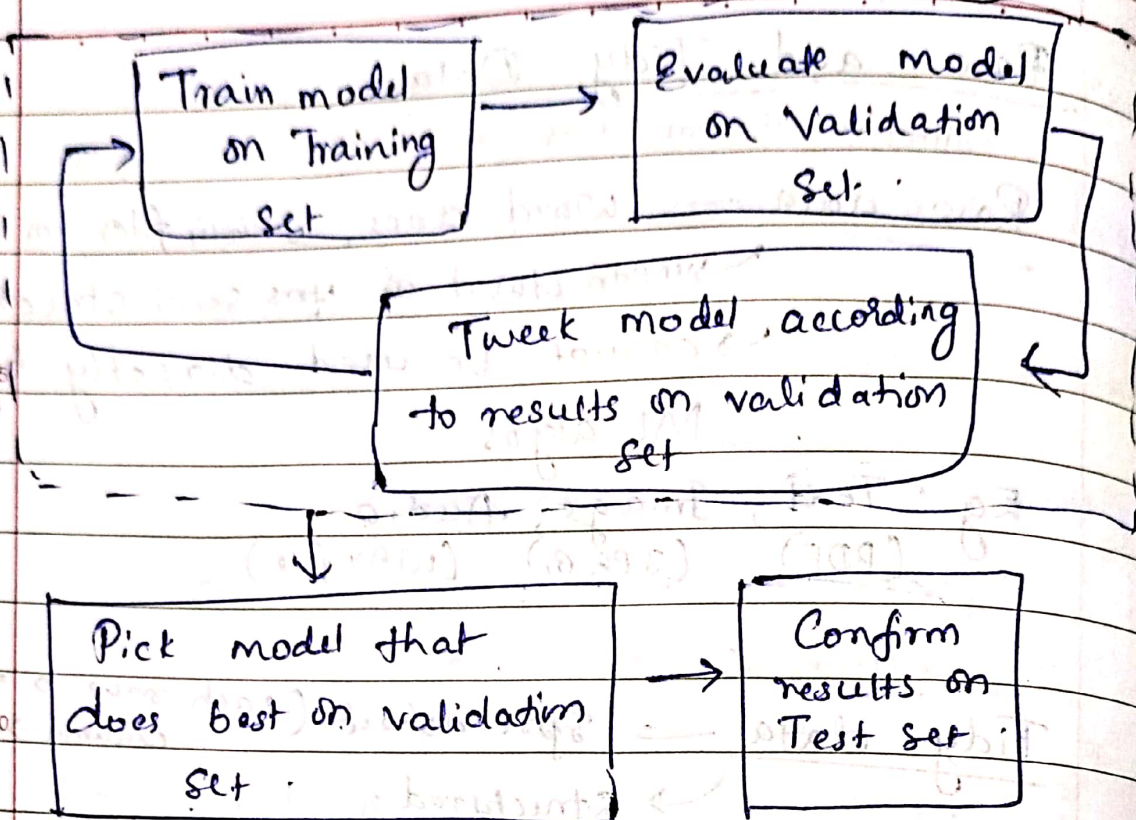
each row → one email

each column → one attribute.

email_length	num_links	has_offer	spam
120	3	yes	Yes
100	2	no	No

Training and Holdout Sets





Validation Set Utility:-

Validation set (or development set) is used to compare diff. algos of diff. configuration of the same algo.

→ It is primary tool for parameter tuning.

Test Set & Generalisation :-

Test set is used at the very end of the project for final performance reporting to the client or product owner.

Success is measured by seeing how well the model predicts the unseen data.

* crowdsourcing → services like Amazon Mechanical Turk can be used to gather an "ensemble" of human predictions to establish achievable accuracy.

Parameters vs Hyperparameters

Parameters → are internal variables that define the model & are directly modified by the algorithm during training.

Hyperparameters → are external points to the algorithm or pipeline that cannot be learned from data.

They are set by the analyst to control the learning process.

Parameters are optimized on the Training Set, Hyperparameters are evaluated on the Validation Set.

Model-based vs Instance-based Learning

> Eg: SVM (support vector machine), Logistic regression, CNN.

> Eg: kNN

Model-based ML algos. take a collection of training examples as i/p & o/p a standalone statistical model.

Once the model is built, the original training data can be discarded, as only

the model is required for the future prediction.

These algos generally seek to find a hypersurface or line that separate classes or follows examples closely.

Instance-based learning algos use the entire dataset as the model, rather than learning a set of fixed parameters.

→ Slower at prediction time than model-based ones.

Shallow vs Deep Learning

Shallow learning algos are defined by learning model parameters directly from the features of training examples.

These models are typically faster to train & require fewer computational resources than deep neural network.

Use shallow models when :

- Dataset is small or medium sized
- Features are well engineered.
- Interpretability is important
- Limited compute resources

Deep neural networks are characterized by having more than one hidden layer b/w i/p & o/p.

Eg: CNN & Transformers used for sequence data

Image classification (CNNs)

Speech recognition.

Machine translation.

Chatbots & LLMs.

Use deep networks when :

- large amounts of data are available.
- Raw data is there.
- Lots of computing power.

Transfer Learning

freeze the 1st few layers & unfreeze the last 2-3 layers & train on new dataset.

Training vs Scoring

Scoring (often called inference) consists of applying a previously trained model to new inputs to generate prediction.

This is runtime process.

Scoring logic must often include error handling, to ensure the system does not look stupid if it encounters an unexpected i/p.

When to use ML :

- 1) - When the problem is too complex for coding.
- 2) - When the problem is constantly changing.
- 3) - When it is a perceptive problem.
- 4) - When the phenomenon is not well studied.
- 5) - When the problem has a simple objective.
- 6) - When it is cost-effective.

1) The problem has too many rules & exceptions.
A partial but useful soln. is acceptable.
I/p's have many correlated attributes.
Eg: Loan repayment prediction.

2) Eg: Web data extraction (scraping)
Fraud detection.

Rule based systems require constant updates.

3) The task involves human-like perception.
I/p's are high-dimensional & raw, Feature design by hand is impractical.
Eg: Image recognition.
Speech recognition.

4) There is no clear scientific or mathematical model.

Eg: Personalized medicine.

System & network logs.

Predicting human behaviour.

The phenomenon is observable but poorly understood.

- 5) different from rule based.
Use ML when $\rightarrow$ yes/no decision.
classification problem.
single numerical prediction

Eg: Spam detection.

Credit approval.

- 6) Eg: Automated spam detection.
Recommendation system.

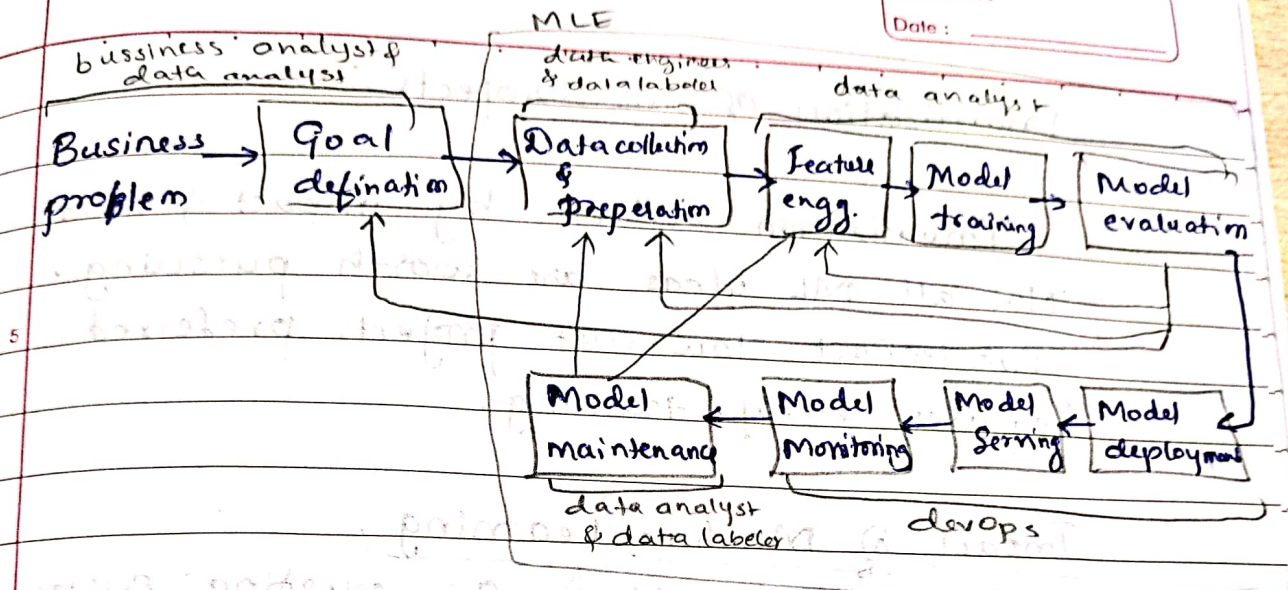
Machine Learning Engineering

ML + software engineering

MLE is essentially Applied Machine Learning

MLE is the application of scientific principles, tools & techniques from both ML+SE to design & build complex computing systems.

ML Project Life cycle: (diagram)



MLE vs MLOPS.
 MLE: building brain of system.
 MLOPS: keeping the brain alive, updated, & healthy in production.

MLOps (ML operations) is the process of automating machine learning using DevOps methodologies such as CI & CD.

Organizing Machine Learning Projects (chapter 2)

Before the project starts -

- Team size & resources depend on goal clarity.
- Unknowns : achievable quality, data volume, feature needs.
- Clear project goal required before execution.
- Complexity estimation is uncertain in ML.

(2.1) Prioritization of ML Projects:

- Prioritization driven by impact & cost.
- Not all ML ideas are worth pursuing.
- High-impact, low-cost projects preferred.
- Requires early training.

(2.1.1) Impact of Machine Learning:

A complex part of an existing system can be rule-based.

The Cost Impact Matrix:

Action Priority Matrix

		Effort	
		Low effort	High effort
Impact	High impact	Quick wins ①	Major projects ②
	Low impact	Fill-ins ③	Thankless tasks ④

(-time, money etc)

Existing rule-based system:-

A company receives thousands of customer support tickets daily. Each ticket must be correctly mapped to customer.

↳ For this ML is used.

↳ This is high impact, low cost.

This is high impact, low cost.

2.1.2 Cost of Machine Learning

3 factors highly influence the cost of a ML project.

- The difficulty of the problem.
- The cost of data.
- The need for accuracy.

2.2 Estimating Complexity of a ML Project

There is no way of estimating with surety.
Best to compare with literature!

1. The Unknowns → look into past works, literature, project.
2. Simplifying the problem.

15 **eg: prblm: Classifying a set of docs into 1000 projects.**
↳ Simplify it by considering 10-100 topics at first, gain confidence, then move into the main project work.

3 Nonlinear progress

- ML project progress is nonlinear.
- Prediction error drops quickly at the start.
- Improvements slowdown over time.

2.3 Defining the Goal of a ML Project

What a model can do?

- Automate
- Alert/prompt
- Organize
- Annotate
- Extract
- Recommend
- Classify
- Quantify
- Synthesize
- Answer questions
- Transform input
- Detect novelty/anomaly

If a business problem cannot be expressed in one of these problems $\rightarrow$ ML likely not the right solution.

Properties of a Successful Model

Read textbook $\rightarrow$

2.4 Structuring a Machine Learning Team

2.4.1 Two Cultures of ML Teams: (Given use case, we need to be able to differentiate or find out these cultures)

Culture 1: Specialists Collaborate

* Team composition

- Data analysts / scientists
- Software engineers

* Expectation

- Data analysts $\leftarrow$ strong ML theory
Model design & experimentation
- Software engineers $\leftarrow$ Production-quality code
Scalable systems

* Shared requirement = common vocabulary
 * Pros

- Deep expertise
- Faster theoretical solns.

* Cons

- Integration challenges
- Scientist solns may be impractical.

Culture 2 : Hybrid Engineers.

* Team composition.

- Engineers with : - ML skills
- S/w engg. skills

* Belief

- One person owns model + code.

* Pros

- Easier production deployment.
- Less rewriting of code.

* Cons

- Hard to hire.
- Shallower expertise in each area.

Members of a ML Team :

* ML engineer - decides what to learn
 Data - ensures data flow correctly
 Software - ensures the system works at scale.

Data labeling expert

Domain expert

MLOps engineers